We've already seen `strcmp()` in the previous section, when we looked at the concept of binary safety (and proved that `strcmp()` is binary safe). `strcasecmp()` is a case-insensitive form of `strcmp()`, `strnatcmp()`, and `strnatcasecmp()`, using natural sorting (also demonstrated in Chapter 5, "Arrays"). In all cases, the functions work by comparing two input strings, returning a negative integer if the first string is less than the second, `0` if both are equal, and a positive integer if the first string is greater than the second. The devil is in the detail, though. Here's an example, using good old British motorways, just like when we looked at array sorting.

```
var_dump(strcmp("M1", "M4"));
var_dump(strcmp("M4", "M11"));
var_dump(strcmp("M1", "M11"));
var_dump(strcmp("M1", "M18"));
var_dump(strcmp("M11", "M18"));
```

```
Output:
int(-3)
int(3)
int(-1)
int(-1)
int(-7)
```

The example above serves to illustrate a couple of things. First, notice that the comparison does not use natural sorting: the second line compares the strings `"M4"` and `"M11"` and returns `3`, indicating that `"M4"` is "greater than." Comparison in this way orders the motorways like this: M1, M11, M18, M4… but *4* is less than *11* or *18*, right? The second thing to notice is the return values, which have a direction (positive, negative) and magnitude. `"M1"` and `"M4"` are three apart, and `"M11"` and `"M18"` are seven apart, but `"M1"` and `"M18"` are one apart! What gives?

The answer lies in how `strcmp()` works internally: it is performing a simple binary comparison. `strcmp()` traverses the byte arrays of both strings and compares values in each position and applies the following logic:

- If bytes don't match, compare byte values, return the difference (return *byte-1–byte-2*).
- If bytes match, continue to next position.